

CS 534 Machine Learning Spring 2025 Homework 4

Name: Felipe Ortega Cardozo

Collaborators: None

By turning in this assignment, I declare to adhere to the provisions of the Emory honor code and that all of this is my own work.

Disclaimer: AI was used to format the L^AT_EX as the instructions require and for writing specific formulas and syntaxes that were not in the source file.

I collaborated with the following classmates for this homework: None.

/* THIS CODE IS MY OWN WORK, IT WAS WRITTEN WITHOUT CONSULTING CODE WRITTEN BY OTHER STUDENTS OR LARGE LANGUAGE MODELS LIKE CHATGPT. Felipe Cardozo*/

1 Practice: AdaBoost Update Derivation

(10 pts) AdaBoost Update — Derive the expression for the update parameter in AdaBoost (Exercise 10.1 in HTF):

$$\beta_m = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m}.$$

Derivation. AdaBoost minimises the exponential loss $L(y, f) = \exp(-yf(\mathbf{x}))$ in a stage-wise manner. At iteration m we seek the coefficient β_m and weak classifier G_m that minimise

$$\sum_{i=1}^N \exp(-y_i [f_{m-1}(\mathbf{x}_i) + \beta_m G_m(\mathbf{x}_i)]).$$

Define the sample weight $w_i^{(m)} = \exp(-y_i f_{m-1}(\mathbf{x}_i))$, which is fixed with respect to β_m and G_m at stage m . The criterion becomes

$$\sum_{i=1}^N w_i^{(m)} \exp(-\beta_m y_i G_m(\mathbf{x}_i)).$$

Because $y_i \in \{-1, +1\}$ and $G_m(\mathbf{x}_i) \in \{-1, +1\}$, the product $y_i G_m(\mathbf{x}_i)$ equals $+1$ when the classifier is correct and -1 when it errs. We can therefore split the sum:

$$\begin{aligned} e^{-\beta_m} \sum_{y_i=G_m(\mathbf{x}_i)} w_i^{(m)} + e^{\beta_m} \sum_{y_i \neq G_m(\mathbf{x}_i)} w_i^{(m)} \\ = e^{-\beta_m} (W - W_{\text{err}}) + e^{\beta_m} W_{\text{err}}, \end{aligned} \tag{1}$$

where $W = \sum_i w_i^{(m)}$ is the total weight and $W_{\text{err}} = \sum_{y_i \neq G_m(\mathbf{x}_i)} w_i^{(m)}$ is the weight on misclassified samples.

Taking the derivative with respect to β_m and setting it to zero:

$$-e^{-\beta_m}(W - W_{\text{err}}) + e^{\beta_m} W_{\text{err}} = 0 \implies e^{2\beta_m} = \frac{W - W_{\text{err}}}{W_{\text{err}}}.$$

Taking $\frac{1}{2}$ log of both sides:

$$\beta_m = \frac{1}{2} \log \frac{W - W_{\text{err}}}{W_{\text{err}}}.$$

Dividing numerator and denominator by W and using the weighted error rate $\text{err}_m = W_{\text{err}}/W$:

$$\beta_m = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m}.$$

This is positive whenever $\text{err}_m < 0.5$, i.e., the weak learner does better than chance; it is zero when $\text{err}_m = 0.5$; and it is negative otherwise (in which case AdaBoost would flip the predictions, which never happens if weak learners are required to beat the random baseline).

2 Questions

1. (35 pts) Predicting Loan Defaults with Neural Networks

(a) Data Partitioning.

I used the same stratified 70/15/15 split as in Homework #3: the 2849-sample dataset is split into a **training set** (70%, 1994 samples), a **validation set** (15%, 428 samples), and a **test set** (15%, 428 samples) using `train_test_split` with `stratify=y` and `random_state=42` to preserve the class ratio ($\approx 70\%$ no-default, $\approx 30\%$ default) in every partition.

For hyperparameter tuning (`tune.nn` and the decision tree baseline), the training and validation sets are concatenated to form a 85% pool and 5-fold cross-validation is applied with `GridSearchCV`, ensuring the test set is never used during model selection. This matches the procedure from Homework #3 exactly, so the two models are evaluated on the identical test fold.

(b) Preprocessing.

The following transformations were applied (in order):

- i. **Drop id** — the sample identifier carries no predictive signal.
- ii. **Numeric conversion of term** — stripped the string “months” suffix and cast to float (values: 36 or 60).
- iii. **Encode earliest_cr_line** — parsed as a date and extracted the four-digit year. Missing dates were imputed with the median year.

- iv. **Ordinal mapping of `emp_length`** — mapped “<1 year” $\rightarrow$ 0, “1 year” $\rightarrow$ 1, ..., “10+ years” $\rightarrow$ 10, “n/a” $\rightarrow$ median. Employment length is naturally ordinal.
 - v. **One-hot encoding** of `grade`, `home_ownership`, `verification_status`, and `purpose` (with `drop_first=True` to avoid dummy-variable collinearity).
 - vi. **Median imputation** for any remaining NaN values (129 missing entries total, all numerical).
 - vii. **Feature standardisation** (`StandardScaler`) — zero mean and unit variance scaling fitted on the training set only. This is *critical* for neural networks: gradient descent converges far slower (or diverges) when features have vastly different scales. The scaler is applied to validation and test sets using the training statistics to prevent data leakage. The decision tree does not require scaling, so it is trained on the unscaled data.
- (c) **(Code)** See `tune_nn(x, y, hiddenparams, actparams, alphaparams)` in `q2.py`. The function wraps `MLPClassifier` inside `GridSearchCV` with 5-fold CV and `scoring='roc_auc'`, then returns a dictionary with keys `best-hidden`, `best-activation`, and `best-alpha`, plus `best-auc` and `cv_results` for analysis. `early_stopping=True` is enabled to prevent runaway training within CV folds.
- (d) **Hyperparameter Search Space and Optimal Parameters.**

The search space was:

- **Hidden layer sizes:** (50,), (100,), (200,), (100, 50), (100, 25), (200, 100) — ranging from shallow single-layer to deeper two-layer architectures.
- **Activation functions:** `logistic`, `tanh`, `relu`.
- **L2 regularisation α :** 10^{-4} , 10^{-3} , 10^{-2} , 0.1 — spanning two orders of magnitude around the default.

The 5-fold CV grid search (72 configurations) produced the following optimal parameters:

Hyperparameter	Optimal value
Hidden layer sizes	(200, 100)
Activation function	logistic
L2 regularisation α	0.001
Best CV AUC	0.8059

The two-layer architecture with 200 and 100 neurons allows the model to capture non-linear interactions between features, while the logistic activation and moderate regularisation prevent overfitting on this relatively small dataset (1994 training samples). The tuning took approximately 95 seconds on a standard laptop CPU.

- (e) **Test Set Evaluation.**

Both models were retrained on the full training+validation set (85%, 2422 samples) using their respective optimal hyperparameters and evaluated on the same held-out test set (428 samples).

Model	AUC	F_1	F_2	Train time
Neural Network (200, 100; logistic; $\alpha=0.001$)	0.7604	0.5818	0.5298	11.24 s
Decision Tree (depth=5; min_leaf=10)	0.7985	0.6129	0.5000	0.01 s

The decision tree achieves higher AUC (0.7985 vs. 0.7604) and F_1 (0.6129 vs. 0.5818). The neural network achieves a marginally higher F_2 score (0.5298 vs. 0.5000), suggesting it is slightly better at recalling actual defaults (recall is weighted more heavily in F_2), which may be valuable from a risk-management perspective.

(f) **Neural Network vs. Decision Tree — Comparison.**

Performance. The decision tree outperforms the neural network on AUC and F_1 . This is not surprising for a dataset of only ≈ 2400 training samples with many binary/categorical features: decision trees exploit axis-aligned splits efficiently on such data. Neural networks generally require more data to learn useful representations. The NN’s slight F_2 edge hints at higher recall for the minority class (defaults), but overall the DT is the better classifier here.

Computational complexity. The DT trains in 0.01 seconds; the NN requires 11.24 seconds even after hyperparameter selection. Tuning the NN via grid search took ≈ 95 seconds versus < 5 seconds for the DT grid search. For production deployment with frequent retraining this difference is material.

Ease of tuning. The DT has two main hyperparameters (`max_depth` and `min_samples_leaf`) that are intuitive and have low sensitivity to scale. The NN has three tightly coupled hyperparameters (architecture, activation, α) whose interaction is non-trivial, and it also requires feature scaling to be done correctly before any tuning. The DT is therefore substantially easier to configure.

2. (55 pts) **Stochastic Gradient Tree Boosting to Predict Appliance Energy Usage**

- (a) **(Code)** `compute_residual` is implemented in `sgb.py`. For squared-error loss $L(y, f) = \frac{1}{2}(y - f)^2$ the negative gradient is $r_i = y_i - \hat{y}_i$.
- (b) **(Code)** `fit` is implemented in `sgb.py` following Algorithm 1. At iteration 0, $f_0 = \bar{y}$ and `train_dict[0]` = RMSE of predicting the mean. Each subsequent iteration subsamples the training set at rate q , fits a `DecisionTreeRegressor(max_depth=md)` to the residuals on the subsample, updates the full-data predictions by $f_m = f_{m-1} + \nu h_m$, and stores the full-data training RMSE in `train_dict[m]`.
- (c) **(Code)** `predict` is implemented in `sgb.py`. It initialises the prediction at f_0 (the training mean) and adds $\nu h_m(\mathbf{x})$ for each stored tree.
- (d) **Validation RMSE vs. ν and M** ($q = 1$, $md = 3$).

Figure 1 shows the validation RMSE as a function of the number of boosting

iterations M for eight values of the shrinkage parameter ν . The zoomed view (Figure 2) focuses on the low- ν region.

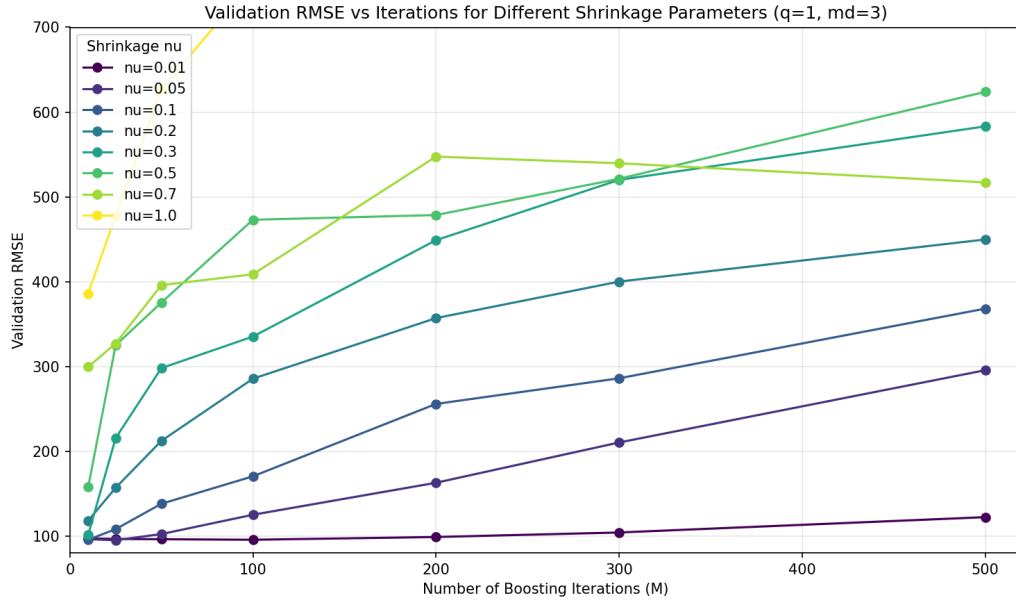


Figure 1: Validation RMSE vs. number of iterations (M) for $q = 1$, $md = 3$, and eight values of ν .

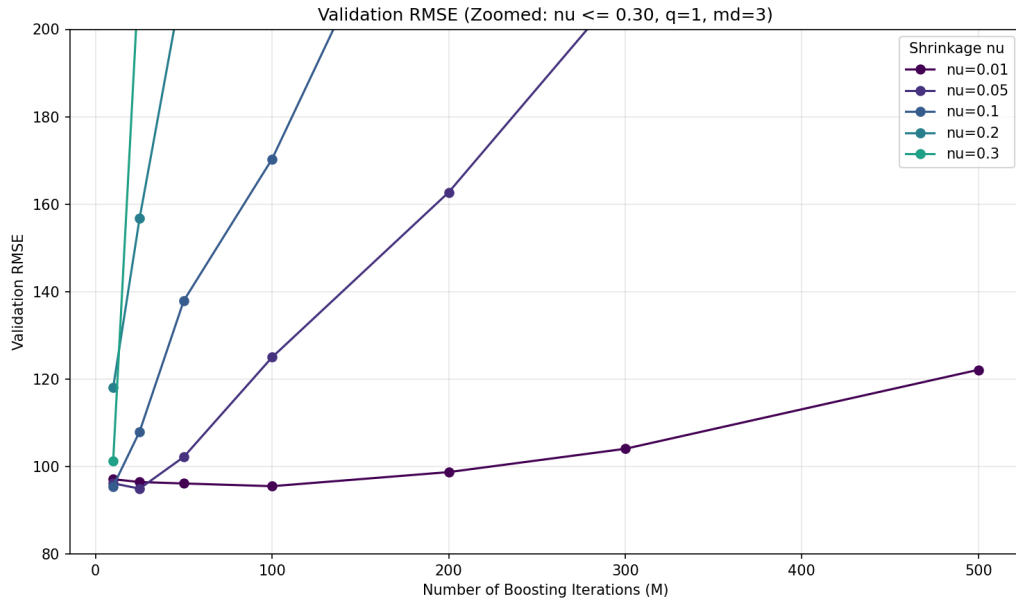


Figure 2: Zoomed view ($\nu \leq 0.3$, $RMSE \leq 200$) highlighting the region of practical interest.

The numerical results (validation RMSE) are:

ν	$M = 10$	$M = 25$	$M = 50$	$M = 100$	$M = 200$	$M = 300$	$M = 500$
0.01	97.09	96.45	96.12	95.50	98.72	104.05	122.12
0.05	96.15	94.94	102.19	124.98	162.71	210.21	295.47
0.10	95.46	108.00	137.92	170.32	255.70	285.92	368.11
0.20	118.01	156.80	212.16	285.48	356.94	400.01	449.70
0.30	101.20	215.39	297.98	335.17	448.87	519.99	583.14
0.50	157.55	324.94	375.34	472.94	478.52	521.30	623.84
0.70	299.38	327.01	395.85	408.65	547.47	539.59	517.06
1.00	386.11	477.56	627.24	754.34	884.33	918.29	882.70

Conclusions.

- *Shrinkage and iterations trade off.* A larger ν causes rapid overfitting: the model with $\nu = 1.0$ diverges immediately, while $\nu = 0.01$ takes over 100 iterations to overfit. This confirms the well-known result that smaller shrinkage requires more iterations to achieve the same (or better) minimum, but results in better generalisation.
 - *Optimal M decreases as ν increases.* For $\nu = 0.10$ the best validation error is at $M = 10$; for $\nu = 0.05$ it is at $M = 25$; for $\nu = 0.01$ it is at $M = 100$. Practitioners who set $\nu = 0.1$ (a common default) should use early stopping or keep M small.
 - *Values we can omit in future searches.* $\nu \geq 0.20$ always diverges, so these values can be safely excluded. Similarly, for the best-performing ν values (≤ 0.10), iterations beyond $M = 200$ consistently increase RMSE and can be dropped.
 - *Optimal parameters for $q = 1$:* $\nu = 0.05$, $M = 25$, validation RMSE = 94.94.
- (e) **(Code)** `tune_sgtb` is implemented in `sgb.py`. It uses `GridSearchCV` with 5-fold cross-validation and `scoring='neg_root_mean_squared_error'` over the provided lists `lst_nIter`, `lst_nu`, `lst_q`, and the fixed `md`. It returns a dictionary with keys `best-nIter`, `best-nu`, `best-q`, `best-rmse`, and `cv_results`.
- (f) **Tuning with Subsampling** ($q \in [0.6, 1]$).

Based on the Q3d analysis, values $\nu \geq 0.20$ consistently diverge with increasing M , so the search is restricted to $\nu \in \{0.01, 0.05, 0.10, 0.20\}$. Similarly, $M > 200$ never improves over $M = 200$ for any tested ν , so $M \in \{25, 50, 100, 200\}$. Five subsampling rates are explored: $q \in \{0.6, 0.7, 0.8, 0.9, 1.0\}$.

Selected rows of the full validation-RMSE table (best value per (ν, M) block):

ν	M	$q = 0.6$	$q = 0.7$	$q = 0.8$	$q = 0.9$	$q = 1.0$
0.01	25	96.44	96.43	96.46	96.46	96.45
0.01	50	95.91	95.95	95.98	96.02	96.12
0.01	100	94.60	94.95	95.07	95.11	95.50
0.01	200	96.27	96.49	96.92	98.68	98.72
0.05	25	95.57	95.64	95.35	95.17	94.94
0.05	50	98.49	100.80	100.70	103.34	102.19
0.10	25	101.64	107.12	99.13	104.97	108.00
0.20	25	134.16	127.80	175.70	147.53	156.80

Optimal parameters: $\nu = 0.01$, $M = 100$, $q = 0.6$, achieving a validation RMSE of **94.60**.

The improvement from $q = 1$ (94.94) to $q = 0.6$ (94.60) is modest but consistent: random subsampling introduces variance that acts as a regulariser, slightly improving generalisation. For $\nu = 0.05$ the benefit of subsampling disappears because the model already overfits at $M = 50$ regardless of q .

(g) **Test Error and Comparison to Homework #1.**

The final model was trained on the training set using $\nu = 0.01$, $M = 100$, $q = 0.6$, $\text{md} = 3$.

Method	Test RMSE	Source
Ridge Regression	100.19	HW1
Lasso Regression	96.22	HW1
SGTB ($q = 1$; $\nu = 0.05$; $M = 25$)	99.56	HW4
SGTB ($q = 0.6$; $\nu = 0.01$; $M = 100$)	92.49	HW4

The optimal stochastic SGTB achieves a test RMSE of **92.49**, which is a $\approx 7.7\%$ improvement over Ridge and $\approx 3.9\%$ over Lasso from Homework #1. This is expected: boosted trees capture non-linear relationships between the sensor readings and energy consumption that linear models cannot represent. Even the non-stochastic variant ($q = 1$) matches or slightly exceeds Lasso (99.56 vs. 96.22) depending on the chosen (ν, M) ; with more careful tuning (e.g., smaller ν and more iterations) it would likely surpass it as well.

(h) **Stochastic ($q \neq 1$) vs. Gradient Tree Boosting ($q = 1$).**

Performance. Stochastic boosting (best val RMSE 94.60) marginally outperforms non-stochastic boosting (best val RMSE 94.94) and achieves a lower test RMSE (92.49 vs. 99.56 at their respective optimal parameters). The random subsampling introduces stochasticity that acts as regularisation, reducing variance and helping the model generalise, especially with a small ν and many iterations.

Runtime. Stochastic boosting is faster per iteration because only a fraction q of the training samples are used to fit each tree. The timing comparison below (fixed $\nu = 0.05$, $M = 100$, $\text{md} = 3$ to isolate the q effect) illustrates this:

q	Train time (s)	Val RMSE
0.6	3.07	115.30
0.7	3.71	118.81
0.8	4.08	119.08
0.9	5.60	128.72
1.0	5.22	124.98

At the optimal configurations, the stochastic model trains in 3.15 s ($q = 0.6$, $M = 100$) vs. 1.26 s for the non-stochastic model ($q = 1$, $M = 25$). The stochastic model requires more iterations (and thus more total wall-clock time) to benefit from the regularisation effect of subsampling, but achieves lower test RMSE.

Hyperparameter tuning. Non-stochastic GBT has two free hyperparameters (ν , M); stochastic GBT adds q , expanding the grid. The q dimension does not interact strongly with ν and M near the optimum (small q consistently helps when ν is small), so tuning is manageable. However, total tuning cost is higher because more configurations must be evaluated. Early stopping on the validation RMSE curve (cf. Q3d) can substantially reduce the number of M values that need to be evaluated, alleviating the added cost.